

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №6

По дисциплине: «Естественно-языковой интерфейс ИС»

Тема: «Разработка автоматизированной системы диалогового взаимодействия с пользователем на естественном языке»

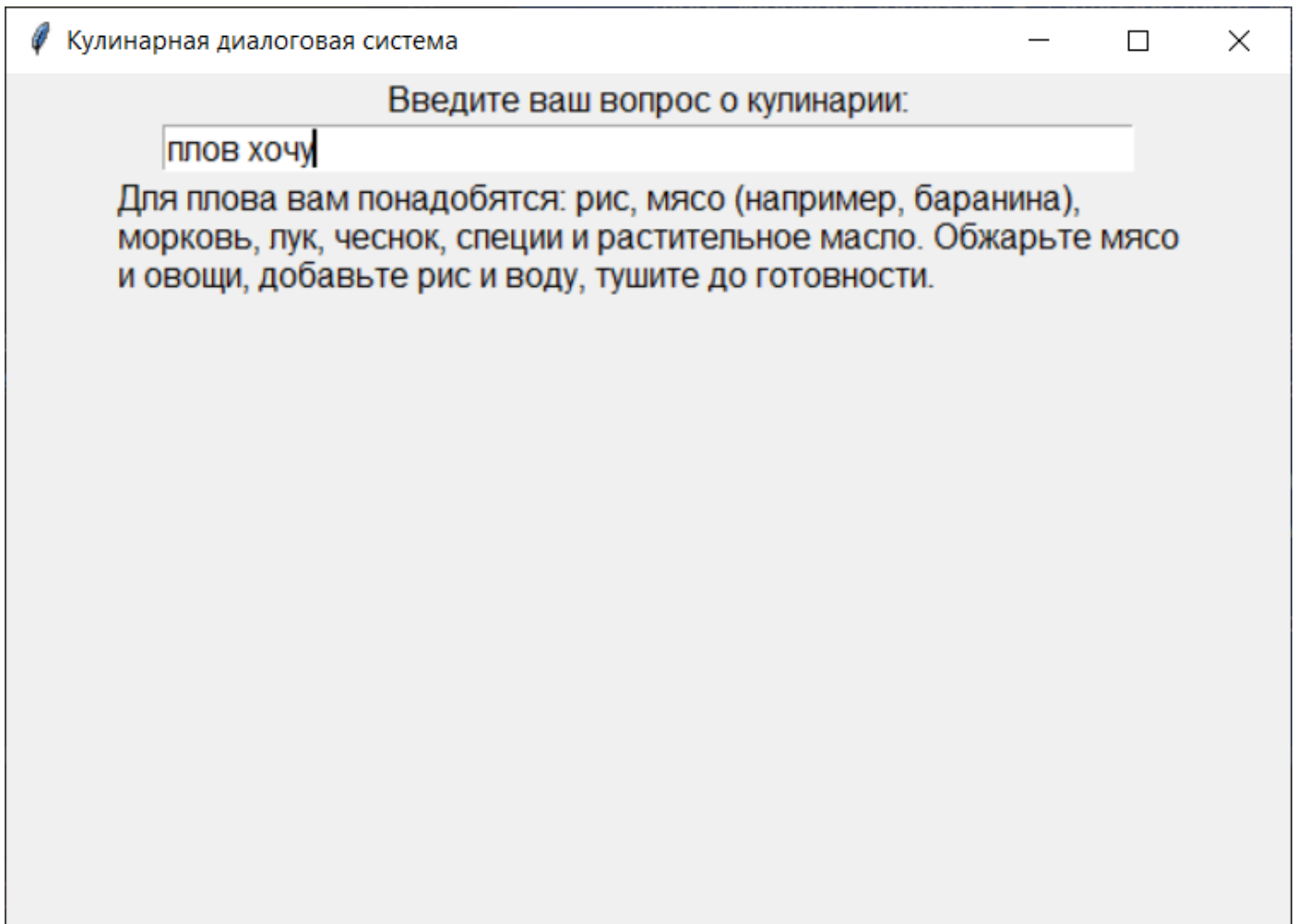
Выполнил:
Студент 3 курса
Группы ИИ-23
Макаевич Н.Р.

Проверил:
Булей Е.В.

Ход работы

Задание:

1. *Входные данные* – текстовое сообщение на заданном естественном языке
2. *Выходные данные* – автоматическая реакция системы на входное сообщение на естественном языке путем формирования ответного сообщения согласно варианту задания
3. Взаимодействие с пользователем посредством графического интерфейса



Код программы:

```
import tkinter as tk
from Levenshtein import distance
from collections import Counter

pairs = [
    ['популярные блюда белорусской кухни', ['В белорусской кухне популярны такие блюда, как драники, колдуны, борщ, мачанка и кулага.']],
    ['рецепт драников', ['Для приготовления драников вам понадобятся: картофель, лук, яйцо, соль и растительное масло. Натрите картофель на терке, добавьте мелко нарезанный лук, яйцо и соль. Обжарьте на сковороде до золотистой корочки.']],
    ['как приготовить борщ', ['Для борща вам понадобятся: свекла, капуста, картофель, морковь, лук, мясо (например, говядина), томатная паста, соль и специи. Сначала сварите мясо, затем добавьте овощи и томатную пасту. Варите до готовности.']],
    ['лучшие блюда итальянской кухни', ['Итальянская кухня славится своими пиццами, пастами, ризотто, лазаньей и тирамису.']],
    ['рецепт пасты карбонара', ['Для пасты карбонара вам понадобятся: спагетти, бекон, яйца, пармезан, чеснок, соль и перец. Обжарьте бекон с чесноком, смешайте с вареными спагетти и соусом из яиц и пармезана.']],
    ['как приготовить суши', ['Для суши вам понадобятся: рис для суши, нори, свежая
```

```

рыба (например, лосось), авокадо, огурец и рисовый уксус. Приготовьте рис, нарежьте
рыбу и овощи, заверните в нори.']],
    ['популярные десерты', ['Популярные десерты включают чизкейк, тирамису, эклеры,
макаруны и брауни.']],
    ['рецепт шоколадного торта', ['Для шоколадного торта вам понадобятся: мука, сахар,
какао, яйца, масло и разрыхлитель. Смешайте ингредиенты, выпекайте в духовке при 180°C
30-40 минут.']],
    ['как приготовить плов', ['Для плова вам понадобятся: рис, мясо (например,
баранина), морковь, лук, чеснок, специи и растительное масло. Обжарьте мясо и овощи,
добавьте рис и воду, тушите до готовности.']],
    ['лучшие блюда французской кухни', ['Французская кухня известна своими круассанами,
рататуем, утиным конфи, луковым супом и фуа-гра.']],
]

```

```

class ChatApplication:
    def __init__(self, master):
        self.master = master
        master.title("Кулинарная диалоговая система")

        self.label = tk.Label(master, text="Введите ваш вопрос о кулинарии:",
font=("Arial", 12))
        self.label.pack()

        self.entry = tk.Entry(master, font=("Arial", 12), width=50)
        self.entry.pack()

        self.response_label = tk.Label(master, text="", font=("Arial", 12),
wraplength=500, justify="left")
        self.response_label.pack()

        self.entry.bind("<Return>", self.send_message)

    def calculate_similarity(self, user_message):
        closest_pair = None
        max_similarity = 0

        user_message_words = user_message.lower().split()
        user_message_counter = Counter(user_message_words)
        user_message_length = len(user_message_words)

        for pair in pairs:
            question = pair[0]
            question_words = question.lower().split()

            d = distance(question.lower(), user_message.lower())
            common_words = sum((user_message_counter &
Counter(question_words)).values())

            normalized_distance = d / max(len(question), len(user_message))
            normalized_common_words = common_words / user_message_length

            similarity = 0.6 * (1 - normalized_distance) + 0.4 *
normalized_common_words
            if similarity > max_similarity:
                max_similarity = similarity
                closest_pair = pair

        return closest_pair

    def send_message(self, event):
        user_message = self.entry.get()
        self.entry.delete(0, tk.END)

        closest_pair = self.calculate_similarity(user_message)
        if closest_pair:
            response = closest_pair[1][0]

```

```
        self.response_label.config(text=response)
    else:
        self.response_label.config(text="Извините, не могу найти подходящий
ответ.")

def main():
    root = tk.Tk()
    app = ChatApplication(root)
    root.geometry("600x400")
    root.mainloop()

if __name__ == "__main__":
    main()
```

Вывод: в ходе выполнения лабораторной работы освоил принципы разработки диалоговых систем с поддержкой естественного языка.